

0123456

arr

3014132462380044586

length

7

```
int count=0;
for (int i=0; i<arr.Length;i++)
{
    int num=arr[i]/10;
    int digit=num%10;
    if (digit==0)
        count++;
}
```

i	i < 7	arr[i]	num arr[i]/10	digit num%10	digit == i	count
						0
0	T	30	3	3	F	
1	T	141	14	4	F	
2	T	324	32	2	T	1
3	T	623	62	2	F	
4	T	8004	800	0	F	
5	T	458	45	5	T	2
6	T	6	0	0	F	
7	F					

בסיום: count = 2, מספר האיברים במערך שספרת העשרות שלהם שווה לאינדקס המערך

ב. ספרת העשרות של כל איבר במערך שונה מהאינדקס

0123

arr

131242353464

length

4

ג. ספרת העשרות של כל איבר במערך שווה לאינדקס

0123

arr

101212323434

length

4

"מערך סיסמאות" במערכת ממוחשבת של בנק מסוים הוא מערך מטיפוס שלם בגודל 10 שבו הבנק שומר את עשר הסיסמאות האחרונות של הלקוח (כדי לוודא שהלקוח לא יבחר אותן שוב). הסיסמאות נשמרות לפי הסדר מן החדשה ביותר לישנה ביותר. הסיסמה החדשה ביותר (העדכנית) נמצאת בתא 0 והסיסמה הישנה ביותר נמצאת בתא האחרון במערך. כאשר מתקבלת סיסמה חדשה, הסיסמה הישנה ביותר (הסיסמה שבתא האחרון במערך) נמחקת מן המערך, שאר הסיסמאות זזות ימינה כל אחת לתא שלידה, והסיסמה החדשה מופיעה בתא הראשון (אינדקס 0).

כתבו פעולה בוליאנית חיצונית ששמה getPassword בשפת Java או GetPass בשפת C#, המקבלת מערך סיסמאות מלא – arr, וסיסמה – password, מטיפוס שלם.

אם הסיסמה שהתקבלה היא חדשה (כלומר אינה זהה לשום סיסמה שמופיעה במערך), הפעולה תוסיף אותה למערך arr בתא 0 (לאחר הזזת שאר הסיסמאות תא אחד ימינה) ותחזיר true. אחרת, מערך הסיסמאות יישאר ללא שינוי ויוחזר false.

דוגמה: עבור המערך arr שלפניכם ר- password = 300, המערך יישאר ללא שינוי והפעולה תחזיר false.

0123456789

arr

12231342453001111017779001951234

הסבר: הסיסמה 300 כבר מופיעה במערך arr, באינדקס 3.

דוגמה נוספת: עבור אותו המערך ר- password = 8888, הפעולה תחזיר true, והמערך ייראה כך בתום הפעולה:

0123456789

arr

88881223134245300111101777900195

הסבר: הסיסמה 8888 אינה אחת הסיסמאות שכבר מופיעות במערך.

כתבו פעולה בוליאנית חיצונית ששמה getPassword בשפת Java או GetPass בשפת C#, המקבלת מערך סיסמאות מלא – arr, וסיסמה – password, מטיפוס שלם.

אם הסיסמה שהתקבלה היא חדשה (כלומר אינה זהה לשום סיסמה שמופיעה במערך), הפעולה תוסיף אותה למערך arr בתא 0 (לאחר הזזת שאר הסיסמאות תא אחד ימינה) ותחזיר true. אחרת, מערך הסיסמאות יישאר ללא שינוי ויוחזר false.

```
1 public static void AddAtIndex(int[] arr, int indx, int num) {
2     for (int i = arr.Length-1; i > indx; i--)
3         arr[i] = arr[i-1];
4     arr[indx] = num;
5 }
6
7 int arr[] = new int[] {0,1,2,3,5,6,7,8};
8 AddAtIndex(arr, 4, 4);
9 Print(arr);                // [0,1,2,3,4,5,6,7]
```

כתבו פעולה בוליאנית חיצונית ששמה getPassword בשפת Java או GetPass בשפת C#, המקבלת מערך סיסמאות מלא – arr, וסיסמה – password, מטיפוס שלם.

אם הסיסמה שהתקבלה היא חדשה (כלומר אינה זהה לשום סיסמה שמופיעה במערך), הפעולה תוסיף אותה למערך arr בתא 0 (לאחר הזזת שאר הסיסמאות תא אחד ימינה) ותחזיר true. אחרת, מערך הסיסמאות יישאר ללא שינוי ויוחזר false.

```
1 public static bool GetPass(int[] arr, int password) {
2     foreach (int p in arr)
3     {
4         if (p == password)
5             return false;
6     }
7     for (int i = arr.Length-1; i > 0; i--)
8         arr[i] = arr[i-1];
9     arr[0] = password;
10    return true;
11 }
```

"מערך מעורבל" הוא מערך מטיפוס שלם שהתהליך המתואר לפניכם בוצע בו 30 פעמים:

- מוגרלים שני מספרי אינדקס תקינים (בגבולות גודל המערך).
- הערכים הנמצאים בתאים שמספריהם עלו בהגרלה מתחלפים ביניהם.

הערך: אם בהגרלה עלו שני מספרי אינדקס זהים, ההגרלה לא תיחשב (כלומר היא לא תיספר כחלק מן התהליך המבוצע 30 פעמים).

"מערך מעורבל" הוא מערך מטיפוס שלם שהתהליך המתואר לפניכם בוצע בו 30 פעמים:

- מוגרלים שני מספרי אינדקס תקינים (בגבולות גודל המערך).
- הערכים הנמצאים בתאים שמספריהם עלו בהגרלה מתחלפים ביניהם.

```
Random rnd = new Random();
```

```
rnd.NextDouble();    // 0.73452...
rnd.NextDouble();    // 0.01893...
rnd.NextDouble();    // 0.99987...
```

```
rnd.Next(10);        // 0..9
rnd.Next(1, 11);     // 1..10
rnd.Next(1, 7);      // dice: 1..6
```

```
1 public static int GetRandomIndex(int[] arr)
2 {
3     Random rnd = new Random();
4     return rnd.Next(arr.Length);
5 }
```

```
1 // assume array has 5 positives
2 public static int SumFivePositives(int[] arr) {
3     int sum = 0;
4     int count = 0;
5     int i = 0;
6     while (count < 5)
7     {
8         if (arr[i] > 0)
9         {
10             sum += arr[i];
11             count++;
12         }
13         i++;
14     }
15     return sum;
16 }
```

"מערך מעורבל" הוא מערך מטיפוס שלם שהתהליך המתואר לפניכם בוצע בו 30 פעמים:

- מוגרלים שני מספרי אינדקס תקינים (בגבולות גודל המערך).
- הערכים הנמצאים בתאים שמספריהם עלו בהגרלה מתחלפים ביניהם.

הערך: אם בהגרלה עלו שני מספרי אינדקס זהים, ההגרלה לא תיחשב (כלומר היא לא תיספר כחלק מן התהליך המבוצע 30 פעמים).

כתבו פעולה חיצונית ששמה shuffle בשפת Java או Shuffle בשפת C# המקבלת מערך מטיפוס שלם - arr ומערבלת אותו. הפעולה תדפיס את ערכי התאים לפי הסדר לאחר הערבול (כל ערך בשורה חדשה).

```
1 public static void Shuffle(int[] arr)
2 {
3     Random rnd = new Random();
4     int count = 0;
5     while (count < 30)
6     {
7         int i1 = rnd.Next(arr.Length);
8         int i2 = rnd.Next(arr.Length);
9         if (i1 != i2)
10        {
11            int tmp = arr[i1];
12            arr[i1] = arr[i2];
13            arr[i2] = tmp;
14            count++;
15        }
16        for (int i = 0; i < arr.Length; i++)
17            Console.WriteLine(arr[i]);
18    }
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
1 public class TourPackage
2 {
3     private int id;
4     private int price;
5     private int maxTime;
6     private int maxData;
7     private int extra;
8
9     public TourPackage(int id, int price,
10                        int maxTime, int maxData)
11     {
12         this.id = id;
13         this.price = price;
14         this.maxTime = maxTime;
15         this.maxData = maxData;
16         this.extra = 0;
17     }
18
19     public void SetExtra(int minutes, int usage) {
20         if (minutes > this.maxTime)
21             this.extra += minutes - this.maxTime;
22         if (usage > this.maxData)
23             this.extra += 2 * (usage - this.maxData);
24     }
25 }
```

נתון מערך מלא – packages מטיפוס TourPackage, שבו נתונים על לקוחות שונים (לאחר שכבר חושב ועודכן הסכום לתשלום עבור חריגה מן החבילה הבסיסית בתכונה extra).

ב. (1) כתבו פעולה חיצונית ששמה calculate בשפת Java או Calculate בשפת C#, המקבלת את המערך packages, ומחזירה את כמות הלקוחות שצריכים לשלם תוספת על חריגה מן החבילה הבסיסית.

(2) כתבו פעולה חיצונית ששמה customers בשפת Java או Customers בשפת C#, המקבלת את המערך packages. הפעולה מחזירה מערך מטיפוס שלם שבו מספרי הזיהוי של הלקוחות (id) שצריכים לשלם תוספת על חריגה מן החבילה הבסיסית (גודל המערך הוא לפי כמות האנשים שחרגו). אין חשיבות לסדר ההופעה במערך.

הערה: חובה להיעזר בפעולה שכתבתם בסעיף ב(1).

```
1 TourPackage p1 = new TourPackage(111, 100, 100, 10);
2 TourPackage p2 = new TourPackage(222, 200, 200, 20);
3 TourPackage p3 = new TourPackage(333, 300, 300, 30);
4 TourPackage p4 = new TourPackage(444, 400, 400, 40);
5
6 p2.SetExtra(220, 30);
7 p3.SetExtra(350, 35);
8
9 TourPackage[] packages = new TourPackage[] {p1, p2, p3, p4};
10
11 Print(Calculate(packages)); // 2
12 Print(Customers(packages)); // [222, 333]
```

הערה: חובה להיעזר בפעולה שכתבתם בסעיף ב(1).

```
1 public static int Calculate(TourPackage[] packages)
2 {
3     int count = 0;
4     for (int i = 0; i < packages.Length; i++)
5     {
6         if (packages[i].GetExtra() > 0)
7             count++;
8     }
9     return count;
10 }
11
12 public static int[] Customers(TourPackage[] packages) {
13     int k = Calculate(packages);
14     int[] arrId = new int[k];
15     int count = 0;
16     for (int i = 0; i < packages.Length; i++)
17     {
18         if (packages[i].GetExtra() > 0)
19         {
20             arrId[count] = packages[i].GetId();
21             count++;
22         }
23     }
24     return arrId;
25 }
```

הערה: חובה להיעזר בפעולה שכתבתם בסעיף ב(1).

א. (1) כתבו במחלקה Lesson פעולה פנימית ששמה isLater בשפת Java או IsLater בשפת C#, המקבלת שיעור אחר – other (מטיפוס Lesson). הפעולה מחזירה true אם השיעור הנוכחי מתחיל מאוחר יותר מן הזמן שמתחיל השיעור האחר (other), ואחרת מחזירה false.

```
1 public class Lesson
2 {
3     private int id;
4     private int hh;
5     private int mm;
6     private int duration;
7
8     public bool IsLater(Lesson other) {
9         if (this.hh > other.GetHh())
10             return true;
11         if (this.hh < other.GetHh())
12             return false;
13         return this.mm > other.GetMm();
14     }
15 }
```

הערה: חובה להיעזר בפעולה שכתבתם בסעיף ב(1).

(2) כתבו פעולה חיצונית ששמה Last המקבלת את המערך arr, ומדפיסה את קוד המקצוע (id) ששיעורו התחיל אחרון ביום הלימודים.

הניחו שיש רק שיעור אחד המקיים תנאי זה.

הערה: חובה להיעזר בפעולה שכתבתם בסעיף א(1).

```
1 Lesson[] arr = new Lesson[]
2 {
3     new Lesson(1315, 13, 15, 45),
4     new Lesson(1445, 14, 45, 45),
5     new Lesson(1330, 13, 30, 45),
6     new Lesson(1345, 13, 45, 45)
7 };
8
9 Last(arr); // 1445
10
11 public static void Last(Lesson[] arr)
12 {
13     int latest = 0;
14     for (int i = 1; i < arr.Length; i++)
15     {
16         if (arr[i].IsLater(arr[latest]))
17             latest = i;
18     }
19     Console.WriteLine(arr[latest].GetId());
20 }
```

ב. (1) כתבו פעולה חיצונית ששמה sumDuration בשפת Java או SumDuration בשפת C# המקבלת את המערך arr וקוד של מקצוע – id. הפעולה מחזירה את מספר הדקות סך הכול של כל השיעורים בקוד המקצוע id המתקיימים באותו יום לימודים.
לדוגמה: אם שיעור במקצוע id נמשך 40 דקות ובאותו היום שיעור נוסף באותו מקצוע נמשך 70 דקות, יוחזר 110.

```
1 Lesson[] arr = new Lesson[]
2 {
3     new Lesson(3333, 13, 15, 30),
4     new Lesson(4444, 14, 45, 45),
5     new Lesson(3333, 13, 30, 45),
6     new Lesson(3333, 13, 45, 30)
7 };
8
9 Print(SumDuration(arr, 3333));    // 105
10 Print(SumDuration(arr, 4444));    // 45
11
12 public static int SumDuration(Lesson[] arr, int id)
13 {
14     int sum = 0;
15     for (int i = 0; i < arr.Length; i++)
16     {
17         if (arr[i].GetId() == id)
18             sum += arr[i].GetDuration();
19     }
20     return sum;
21 }
```

(2) כתבו פעולה חיצונית ששמה longest בשפת Java או Longest בשפת C# המקבלת את המערך arr. הפעולה תחזיר את קוד המקצוע (id) שמספר הדקות סך הכול של כל שיעוריו באותו יום לימודים הוא הגדול ביותר. הניחו שיש רק מקצוע אחד המקיים תנאי זה.
הערה: חובה להיעזר בפעולה שכתבתם בסעיף ב(1).

```
1 Lesson[] arr = new Lesson[]
2 {
3     new Lesson(3333, 13, 15, 30),
4     new Lesson(4444, 14, 45, 45),
5     new Lesson(3333, 13, 30, 45),
6     new Lesson(3333, 13, 45, 30),
7     new Lesson(4444, 14, 45, 120)
8 };
9
10 Print(Longest(arr));    // 4444
11
12 public static int Longest(Lesson[] arr)
13 {
14     int maxId = arr[0].GetId();
15     int max = SumDuration(arr, maxId);
16     for (int i = 1; i < arr.Length; i++)
17     {
18         int sum = SumDuration(arr, arr[i].GetId());
19         if (sum > max)
20         {
21             max = sum;
22             maxId = arr[i].GetId();
23         }
24     }
25     return maxId;
26 }
```

נתונה פעולה חיצונית ששמה digitSum בשפת Java או DigitSum בשפת C#, המקבלת מספר שלם – num1 ומחזירה את סכום כל הספרות במספר. אפשר להשתמש בפעולה כלי לממש אותה.
לדוגמה: עבור num1 = 961, הפעולה תחזיר 16 (9 + 6 + 1).
"סכום הספרות העמוק" של מספר כלשהו הוא מספר **חד־ספרתי** המתקבל באופן שלהלן:
מחשבים את סכום הספרות שוב ושוב עד שמתקבל מספר חד־ספרתי.
דוגמאות:
"סכום הספרות העמוק" של המספר 5 הוא 5.
"סכום הספרות העמוק" של המספר 36 הוא 9 (3+6).
"סכום הספרות העמוק" של המספר 942378 הוא 6, כמפורט להלן:
 $9 + 4 + 2 + 3 + 7 + 8 = 33$
 $3 + 3 = 6$

א. (1) כתבו פעולה חיצונית ששמה deepSum בשפת Java או DeepSum בשפת C# המקבלת מספר שלם num1 שאינו שלילי ומחזירה את "סכום הספרות העמוק" שלו.

```
1 public static int DigitSum(int num)
2 {
3     int sum = 0;
4     while (num > 0)
5     {
6         sum += num % 10;
7         num /= 10;
8     }
9     return sum;
10 }
11
12 public static int DeepSum(int num1)
13 {
14     while (num1 > 9)
15         num1 = DigitSum(num1);
16     return num1;
17 }
```


(2) "סכום הספרות העמוק" יכול להיות אי-זוגי (למשל 5 כמו בדוגמה הראשונה שלעיל) או זוגי (למשל 6 כמו בדוגמה השלישית שלעיל). יש הטוענים כי בטווח שבין 1 ל- 999999 יש יותר מספרים בעלי "סכום ספרות עמוק" זוגי ממספרים בעלי "סכום ספרות עמוק" אי-זוגי.
כתבו פעולה חיצונית ששמה isCorrect בשפת Java או IsCorrect בשפת C# המחזירה true אם הטענה נכונה, ואחרת מחזירה false.
הערה: חובה להיעזר בפעולה שכתבתם בסעיף א(1).

```
1
2 IsCorrect(); // false: evens=444444 odds=555555
3
4 public static bool IsCorrect()
5 {
6     int evens = 0;
7     int odds = 0;
8     for (int i = 1; i <= 999999; i++)
9     {
10         if (DeepSum(i) % 2 == 0)
11             evens++;
12         else
13             odds++;
14     }
15     return evens > odds;
16 }
```

נתונה פעולה חיצונית נוספת, ששמה digitExists בשפת Java או DigitExists בשפת C#, המקבלת מספר שלם – num וקסרה – digit. הפעולה מחזירה true אם הספרה digit מופיעה במספר num לפחות פעם אחת, ואחרת מחזירה false. אפשר להשתמש בפעולה בלי לממש אותה.
ב. כתבו פעולה חיצונית ששמה inBoth בשפת Java או InBoth בשפת C# המקבלת שני מספרים שלמים: num1 ו- num2. הפעולה מחזירה true אם "סכום הספרות העמוק" של num1 מופיע במספר num2, וגם "סכום הספרות העמוק" של num2 מופיע במספר num1, אחרת היא מחזירה false.
למשל עבור num1 = 36 ו- num2 = 942378, הפעולה תחזיר true כי "סכום הספרות העמוק" של num1 (9) מופיע במספר num2 (942378), ו"סכום הספרות העמוק" של num2 (6) מופיע במספר num1 (36).
הערה: חובה להיעזר בפעולה שכתבתם בסעיף א(1).

```
1
2 Print(InBoth(36, 942378)); // true
3 Print(InBoth(1111, 2222)); // false
4
5 public static bool DigitExists(int num, int dgt)
6 {
7     while (num > 0)
8     {
9         if (num % 10 == dgt)
10             return true;
11         num /= 10;
12     }
13     return false;
14 }
15
16 public static bool InBoth(int num1, int num2)
17 {
18     return DigitExists(num1, DeepSum(num2)) &&
19         DigitExists(num2, DeepSum(num1));
20 }
```